WaveShare ESP32-S3-Touch-LCD-4.3B Driver Library

ไลบรารี Arduino / PlatformIO สำหรับบอร์ด **Waveshare ESP32-S3-Touch-LCD-4.3B**
ออกแบบมาเพื่อรองรับการใช้งานพอร์ตสื่อสารอุตสาหกรรม **RS485**, **CAN Bus (TWAI)**, **I2C Bus** และชิปขยายขา **CH422G IO Expander** อย่างครบถ้วน สะดวก และมีประสิทธิภาพสูง

อ้างอิงเอกสารอย่างเป็นทางการ: ESP32-S3-Touch-LCD-4.3B | WaveShare Documentation

---

📌 ตารางการเชื่อมต่อฮาร์ดแวร์ (Hardware Pinout & Mapping)

1. Terminal Block P1 (บล็อกเทอร์มินอลขันสายด้านหลัง/ด้านล่าง)

ขา (Pin)	ชื่อสัญญาณ (Label)	คำอธิบายการทำงาน (Description)	พิน ESP32-S3 / วงจร
1	`VIN`	ไฟเลี้ยงบอร์ด DC กว้าง (7V – 36V)	วงจร Step-Down บนบอร์ด
2	`GND`	กราวด์ไฟเลี้ยง (Power Ground)	GND
3	`I2C_VCC`	ไฟเลี้ยงสำหรับเซนเซอร์ I2C (เลือก 3.3V หรือ 5V ผ่านจัมเปอร์)	VOUT
4	`GND`	กราวด์สัญญาณ I2C	GND
5	`D_SDA`	ข้อมูล I2C Data (ภายนอก)	**GPIO 8** (พูลอัปบนบอร์ด)
6	`D_SCL`	สัญญาณนาฬิกา I2C Clock (ภายนอก)	**GPIO 9** (พูลอัปบนบอร์ด)
7	`CANL`	สายสัญญาณ CAN Low	Transceiver `TJA1051T/3`
8	`CANH`	สายสัญญาณ CAN High	Transceiver `TJA1051T/3`
9	`RS485_A`	สัญญาณ RS485 Differential A (D+)	Transceiver `SP3485EN`
10	`RS485_B`	สัญญาณ RS485 Differential B (D-)	Transceiver `SP3485EN`
11	`DOUT0`	Isolated Digital Output 0	ออปโต้คัปเปิลควบคุมภายนอก
12	`DOUT1`	Isolated Digital Output 1	ออปโต้คัปเปิลควบคุมภายนอก
13	`DI_COM`	ขากลางร่วมของ Digital Input (Common)	วงจรออปโต้ขาเข้า
14	`GND`	กราวด์สัญญาณ	GND
15	`DIN0`	Isolated Digital Input 0	ควบคุมผ่าน **CH422G EXIO0**

ขา (Pin)	ชื่อสัญญาณ (Label)	คำอธิบายการทำงาน (Description)	พิน ESP32-S3 / วงจร
16	`DIN1`	Isolated Digital Input 1	ควบคุมผ่าน **CH422G EXIO5**

2. รายละเอียดพอร์ตสื่อสารแต่ละพอร์ต

1) RS485 (Hardware Serial + Auto Direction)

- **ESP32-S3 TX:** `GPIO 44` (ส่งข้อมูล)
- **ESP32-S3 RX:** `GPIO 43` (รับข้อมูล)
- **ชิป Transceiver:** `SP3485EN`
- **การควบคุมทิศทาง (DE/RE): Automatic Direction Control ในระดับฮาร์ดแวร์** ไม่จำเป็นต้องสลับขา RTS ในโค้ด สามารถส่งและรับข้อมูลผ่าน Hardware Serial ได้ทันที
- **ตัวต้านทาน 120Ω:** มีสวิตช์ SW2 บนบอร์ดสำหรับเปิด/ปิด 120Ω Termination Resistor

2) CAN Bus (ESP32-S3 TWAI Controller)

- **ESP32-S3 TX:** `GPIO 15`
- **ESP32-S3 RX:** `GPIO 16`
- **ชิป Transceiver:** `TJA1051T/3`
- **ความเร็ว:** รองรับตั้งแต่ 25 kbps ถึง 1 Mbps (ค่าเริ่มต้นแนะนำ 500 kbps)
- **ตัวต้านทาน 120Ω:** มีสวิตช์ SW2 บนบอร์ดสำหรับเปิด/ปิด 120Ω Termination Resistor

3) I2C Bus & อุปกรณ์ที่ต่ออยู่ร่วมกัน

- **ESP32-S3 SDA:** `GPIO 8`
- **ESP32-S3 SCL:** `GPIO 9`
- **อุปกรณ์ภายในที่ใช้ I2C บัสนี้ร่วมกัน:**
  - **GT911 (Capacitive Touch):** Address `0x5D` (หรือ `0x14`)
  - **CH422G (IO Expander):** Address `0x24` (Config), `0x38` (Output), `0x26` (Input)
  - **PCF85063 (RTC):** Address `0x51` (หากติดตั้งบนบอร์ด)
  - **เซนเซอร์ภายนอก:** ต่อผ่านขั้วขันสาย `D_SDA` (พิน 5) และ `D_SCL` (พิน 6)

4) CH422G IO Expander (ควบคุมหน้าจอ & พอร์ตดิจิทัล)

- **EXIO0:** `DIN0` (อ่านค่า Digital Input 0)
- **EXIO1:** `TP_RST` (รีเซ็ตทัชสกรีน GT911)
- **EXIO2:** `DISP` (เปิด/ปิดไฟหน้าจอ Backlight)
- **EXIO3:** `LCD_RST` (รีเซ็ตจอ LCD ST7701)
- **EXIO4:** `SD_CS` (Chip Select ของช่องใส่ MicroSD Card)
- **EXIO5:** `DIN1` (อ่านค่า Digital Input 1)

🚀 วิธีการใช้งาน (Quick Start)

ตัวอย่างที่ 1: เริ่มต้นระบบทั้งหมดในบรรทัดเดียว (All-in-One Init)

```
#include <WaveShare4_3B.h>

void setup() {
    Serial.begin(115200);

    // เริ่มต้นระบบทั้งหมด:
    // RS485 ที่ 9600 baud | CAN ที่ 500 kbps | I2C ที่ 100 kHz
    // และเปิดไฟหน้าจอ Backlight อัตโนมัติ!
    WS43B.begin(9600, 500000, 100000);
}

void loop() {
    // ส่งข้อมูลออก RS485
    WS43B.rs485.println("Hello RS485");

    // ส่งเฟรม CAN Bus
    uint8_t data[8] = {1, 2, 3, 4, 5, 6, 7, 8};
    WS43B.can.send(0x123, data, 8);

    delay(1000);
}
```

ตัวอย่างที่ 2: การใช้งาน RS485 & Modbus RTU

```
#include <WaveShare4_3B.h>

void setup() {
    Serial.begin(115200);
    WS43B.rs485.begin(9600); // เริ่มต้น RS485 ที่ 9600 baud, 8N1
}

void loop() {
    // 1. อ่านข้อมูลทั่วไป
    if (WS43B.rs485.available()) {
        char c = (char)WS43B.rs485.read();
        Serial.print(c);
    }

    // 2. ส่งคำสั่ง Modbus RTU Read Holding Registers (Function 0x03)
    // อ่าน Slave ID 1, เริ่มต้นที่ Register 0x0000, จำนวน 2 รีจิสเตอร์
    WS43B.rs485.sendModbusReadHolding(1, 0x0000, 2);

    // 3. รับและตรวจสอบ CRC16 Modbus อัตโนมัติ
    uint8_t rxBuf[64];
    size_t rxLen = 0;
    if (WS43B.rs485.receiveModbusFrame(rxBuf, sizeof(rxBuf), rxLen, 200)) {
```

```
        Serial.printf("ได้รับ Modbus Response ถูกต้อง! ขนาดยาว %d ไบต์\n", rxLen);
    }

    delay(2000);
}
```

ตัวอย่างที่ 3: การใช้งาน CAN Bus (TWAI)

```
#include <WaveShare4_3B.h>

void setup() {
    Serial.begin(115200);
    WS43B.can.begin(500000); // 500 kbps
}

void loop() {
    // รับเฟรม CAN
    twai_message_t msg;
    if (WS43B.can.receiveMessage(msg, 10)) {
        WaveShare4_3B_CAN::printMessage(msg, Serial);
    }

    // ส่งเฟรม CAN Standard ID 0x123
    uint8_t payload[4] = {0xDE, 0xAD, 0xBE, 0xEF};
    WS43B.can.send(0x123, payload, 4);

    delay(1000);
}
```

ตัวอย่างที่ 4: สแกนและอ่านเซนเซอร์บนบัส I2C

```
#include <WaveShare4_3B.h>

void setup() {
    Serial.begin(115200);
    WS43B.i2c.begin(100000); // 100 kHz

    // สแกนอุปกรณ์ I2C ทั้งหมด และพิมพ์รายงานออกมา
    WS43B.i2c.scan(Serial);
}

void loop() {
    // เช็คว่าเซนเซอร์ address 0x44 ต่ออยู่หรือไม่
    if (WS43B.i2c.isDeviceConnected(0x44)) {
        uint8_t val;
        WS43B.i2c.readRegister8(0x44, 0x00, val);
```

```
    }
    delay(5000);
}
```

ตัวอย่างที่ 5: ควบคุม Backlight และอ่าน Isolated Digital Inputs

```
#include <WaveShare4_3B.h>

void setup() {
    WS43B.i2c.begin();
    WS43B.io.begin(WS43B.i2c.getWire());

    // ควบคุมไฟหน้าจอ Backlight
    WS43B.io.setBacklight(true);  // เปิดไฟหน้าจอ
    // WS43B.io.setBacklight(false); // ปิดไฟหน้าจอ
}

void loop() {
    // อ่านค่า Digital Input 0 และ 1 จากบล็อกขันสาย
    bool din0 = WS43B.io.readDigitalInput(0); // P1 Pin 15
    bool din1 = WS43B.io.readDigitalInput(1); // P1 Pin 16

    Serial.printf("DIN0: %s | DIN1: %s\n", din0 ? "HIGH" : "LOW", din1 ? "HIGH"
: "LOW");
    delay(500);
}
```

🗁 โครงสร้างตัวอย่างโค้ด (Examples Included)

ในโฟลเดอร์ `examples/` ประกอบด้วยตัวอย่างพร้อมรันใน Arduino IDE:

1. `01_RS485_Echo_Test` : ส่ง-รับและ Echo ข้อมูลผ่าน RS485
2. `02_RS485_Modbus_Read` : รันเป็น Modbus RTU Master พร้อมคำนวณและตรวจสอบ CRC16
3. `03_CAN_Send_Receive` : ส่ง-รับเฟรม CAN (TWAI) ตรวจสอบความเร็วและแพ็กเก็ต
4. `04_I2C_Scanner_And_Sensor` : สแกนหาชิปและเซนเซอร์ที่ต่อผ่าน Terminal D_SDA/D_SCL
5. `05_IO_Expander_And_Digital_IO` : ควบคุม Backlight และอ่านค่า Opto Isolated DI
6. `06_All_In_One_Industrial_Demo` : รวมการทำงานทุกพอร์ต (RS485 + CAN + I2C + DI/Backlight) พร้อมกัน

🛠 การตั้งค่าใน Arduino IDE / PlatformIO

Arduino IDE

1. คัดลอกโฟลเดอร์นี้ไปไว้ที่โฟลเดอร์ `Arduino/libraries/`
2. เลือก Board: **ESP32S3 Dev Module**

3. ตั้งค่าการตั้งค่าบอร์ด:
    - **PSRAM:** "OPI PSRAM"
    - **Flash Size:** "16MB (128Mb)"
    - **USB CDC On Boot:** "Enabled" (เพื่อให้พิมพ์ Serial ออกทางช่อง Type-C ได้สะดวก)
    - **Partition Scheme:** "16M Flash (3MB APP/9.9MB FATFS)" หรือขนาดที่เหมาะสม

PlatformIO (`platformio.ini`)

```
[env:esp32-s3-touch-lcd-43b]
platform = espressif32
board = esp32-s3-devkitc-1
framework = arduino
board_build.arduino.memory_type = qio_opi
board_build.flash_size = 16MB
board_build.psram_type = opi
build_flags =
    -DBOARD_HAS_PSRAM
    -DARDUINO_USB_CDC_ON_BOOT=1
monitor_speed = 115200
```

🛠️ การตั้งค่า Waveshare โมดูล8-ch โมดูลรีเลย์อิเล็กทรอนิกส์ Modbus RTU (D)

อ้างอิงเอกสารอย่างเป็นทางการ: WaveShare Modbus RTU Relay (D) Documentation

คำสั่งตั้งค่า `Address`

```
Set the device address as 0x01: `00 06 40 00 00 01 5C 1B`
Set the device address as 0x02: `00 06 40 00 00 02 1C 1A`
Set the device address as 0x03: `00 06 40 00 00 03 DD DA`
```

คำสั่งอ่าน `Address`

```
For example: [Address 2 device]
Send:    00 03 40 00 00 01 90 1B
Receive: 00 03 02 00 02 04 45       //Address: 0x02
```